Loki III: Reverse Engineering Structural Netlists without a Language Model
How far do provenance, a Bayesian L1 and a graph L2 get on their own?

Peter Jamieson
Electrical and Computer Engineering, Miami University
Oxford, OH, USA
jamiespa@miamioh.edu

***Abstract*—Gate-Breaker showed that a language model can recover functional RTL from a gate-level netlist, and it ended by naming the graph techniques its model needed: clustering to name buses, a classification of signals as datapath or control, and partitioning. Loki III asks the complementary question: how much structure can be recovered *without* a language model, from provenance, a Bayesian hypothesis flow and graph analysis alone? This paper describes the benchmarks built to answer that question and reports the recovery statistics under the Gate-Breaker metrics. The benchmarks are 68 synthesizable micro designs compiled by a provenance-tracked front end into named, labelled, anonymised netlist datasets; a renamed-instance protocol then proves the pipeline naming-neutral across 105,646 renamed identifiers. On the L1 hypothesis flow, the structures that are already reverse engineered by the synthesis flow recover almost perfectly (hard adders 67 of 67, hard multipliers 30 of 30), while the structures the flow erased recover weakly (macro-F1 0.354 on the first floor, 0.108 on the second, calibrated). The L2 graph stage beats a 1-WL baseline on real micros once node, graphlet, WL and NPN features combine (macro-F1 0.4241 against 0.2717) and its clusters carry the designer's hierarchy (Louvain modularity 0.4220). We state the honest limits: no language model step exists, the recovered structure is structural rather than functional, and the paper's own determinism defects are reported.**

***Index Terms*—reverse engineering, netlist, provenance, Bayesian inference, graph neural networks, FPGA CAD**

I. Introduction

Gate-Breaker [**?**] takes a gate-level netlist with obfuscated wire names, formats it into a four-step prompt (interface, decomposition, function, generation), asks a language model to write RTL, and checks the result by simulation. On 156 VerilogEval problems [**?**] its best model recovered functional RTL for 75.1% of the runs, and 53.4% on the hardest quartile. The paper's own conclusion is the interesting part: external graph tools should help the model – clustering to name buses, a classification of signals as datapath or control, and partitioning of the netlist into smaller parts.

Loki III (https://github.com/drpaj12/loki) is the Yggdrasil III suite's reverse-engineering tool, and it targets exactly those three items – with no language model step at all. Its pipeline is L0, a provenance ground-truth dataset made from the benchmarks; L1, a Bayesian hypothesis flow whose pattern detectors propose structures and whose proofs confirm them; and L2, graph analysis (clustering, structural features, graph neural networks) over the same netlists. The research question this paper answers is: *how good are the L0, L1 and L2 experiments, without more intelligence, at recovering structure?* The answer matters for two audiences. For the Gate-Breaker line, it prices the graph side of the future work the paper names. For reverse engineering, it measures what a provenance-tracked front end and honest statistical machinery can do before any model is added.

The contributions are: (1) a benchmark method – provenance-tracked, labelled, anonymised netlist datasets with a renamed-instance protocol that proves naming neutrality, so the pipeline can be evaluated on designs whose names carry nothing; (2) the L1 and L2 recovery statistics under the Gate-Breaker metrics and under Loki's own (precision, recall, calibration, clustering quality, GNN against a 1-WL baseline); (3) the honest limits, including two determinism defects the experiments found.

II. Background

A. The provenance problem and Odin III

Synthesis modifies the designer's functional intention: flattening removes hierarchy, optimization merges operators, and mapping covers logic with cells that carry no source names. Linking each netlist object back to the HDL that produced it is the provenance problem [**?**]. Odin III, the provenance-tracked front end under Loki, keeps that link: every object of its one IR records its source location, and the link survives elaboration, optimization, partial mapping and the linked ABC flow. Loki stands on that link: its L0 datasets carry the answer key the synthesis tool kept, which is what makes the recovery statistics in this paper possible at all.

B. Gate-Breaker as the method baseline

Gate-Breaker measures recovery by functional correctness and by the similarity of the recovered and reference RTL:

Success Rate (the fraction of K samples that simulate correctly), Success@k (the fraction of designs solved within the first k samples), AST similarity (mean subtree Jaccard over normalized identifiers, depth $d = 3$) and CFG similarity (a normalized shortest-path node kernel over control-flow graphs). Its complexity score weights sequential blocks, generates, loops, instantiations, always blocks, buses, multiplexers, cases, arithmetic operators, assignments and concatenations into quartiles Q1 (simple combinational) through Q4 (complex sequential). Loki reports its own statistics on the same axes where they apply – recovery rate per structure kind, calibration error, clustering quality against the known hierarchy, and GNN accuracy against a 1-WL baseline – and the Gate-Breaker metrics on the replication runs.

III. Methodology: the benchmarks and the pipeline

A. L0: provenance ground-truth datasets

The benchmark set is the 68-design micro suite (counters, registers, arithmetic, ALUs, muxes, comparators, small FSMs) that Odin III already compiles and checks against Yosys. L0 compiles each design on the k6 and EArch architectures and on the LPM and cmos6 libraries into floors: `f1` (the mapped VTR netlist), `f2` (the LPM library netlist) and `f3` (the cmos6 gate netlist). Every netlist object carries its provenance (source file and line), a neutral `n`k name from one shared counter, and a label: the structure kind (`construct_type`) is the answer key in `labels.csv`, and the netlist-visible hard-block kind (`cell_type`) travels in the graph labels, so structures that are already reverse engineered stay visible to the reconstruction. `loki check` validates every dataset: 68 designs, 269 floors, 100% labelled rows, zero errors. The two hard-block designs the v1.0 front end refused now compile on the v1.1.0 SystemVerilog front end, so the set is complete.

Renamed instances. A separate protocol asks the naming question directly: rename every identifier of every netlist (105,646 identifiers across 269 netlists) through a shared permutation, run the pipeline on the renamed floors, and inverse-map every token back. The verifier reports token-for-token equality with the original, zero failures, and the base and renamed score digests are byte-identical (sha256 over all run scores, 204 of 204 pairs) – the pipeline is naming-neutral, so nothing it recovers is an artefact of a name.

B. L1: the Bayesian hypothesis flow

L1 is a round-based hypothesis flow over the netlist, with no language model. Pattern detectors (arith, bus, ff_pattern, fsm, mux, role, seqfn, unit, and the `hard` detector of Task 17) propose structures with likelihoods; `cone_check` proofs confirm them by equivalence against the lowered cone; Bayes' rule updates each hypothesis from the evidence; and a confirmed guess commits as a new level of an ordinary Odin III design (`raise`, `submod`), so the result can be written, drawn and traced back to the gates. The `hard` detector reads the `unmap` pass's facts: a hard adder, multiplier or memory that the flow already reverse engineered is recovered directly, with the decline reasons the pass recorded (carry-in chains and the like) kept as open hypotheses. A committed calibration (per-floor weights over the structural, WL and NPN features) turns likelihoods into probabilities; the reported expected calibration error (ECE) is the honest check on that step.

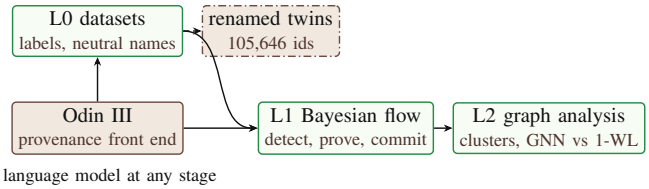

Fig. 1. The Loki III pipeline. L0 compiles the benchmarks through Odin III into provenance-tracked, labelled, anonymised datasets (and their renamed twins). L1 proposes and proves structures in a Bayesian flow that commits into the same IR. L2 reads the graphs back and scores clustering and a GNN against a 1-WL baseline. No stage has a language model.

C. L2: graph analysis

L2 reads the same datasets as graphs: node features from Weisfeiler-Leman colours, NPN signatures and ORCA graphlet counts; region and partition records; four clustering methods (k-means, affinity propagation, spectral, Louvain) scored by modularity and by normalized mutual information against the known module hierarchy; and a graph neural network (GraphSAGE, and GCN as the variant) trained on the labelled graphs with a 1-WL baseline as the honest competitor. The GNN targets the construct type of every node; the question it answers is whether the learned node features beat what one Weisfeiler-Leman round already knows.

IV. Results

A. L1 recovery: what the flow already reverse engineered, and what it erased

Table I reports L1 on the renamed floors (identical to base, as the neutrality check requires). The head-line result is the `hard` detector: hard adders recover 67 of 67 with precision 0.985, hard multipliers 30 of 30, because the synthesis flow itself already reverse engineered them and `unmap` says so. The rest is the honest difficulty: on f1 the calibrated macro-F1 is 0.354 (counted 271, matched 107, ground truth 688), with `ff_en` at 0.364, registers at 0.021 and muxes at 0.018; on f2 it is 0.108 (91/19/686), with adders at F1 0.237 (precision 1.000, recall 0.134) and most other kinds near zero. Without calibration the flow proposes far more and confirms little: f1 macro-F1 0.202 with 1,128 counted (323 of them declined-adder proposals), f2 0.075. Calibration is honest about itself: ECE 0.217 calibrated against 0.650 uncalibrated.

B. L2: clusters that carry the hierarchy, and a GNN that beats 1-WL

Table II scores the four clusterings against the known module hierarchy. Louvain leads (modularity 0.4220, NMI 0.1360); k-means and affinity propagation do not find the hierarchy (negative modularity), which is itself a result: graph structure alone, without the labels, favours the community

TABLE I
L1 RECOVERY ON THE RENAMED FLOORS, PER KIND. PRECISION/RECALL/F1 AGAINST THE L0 GROUND TRUTH; CALIBRATED (MICRO-FIT) COLUMNS. THE HARD KINDS RECOVER FROM UNMAP FACTS; THE REST MUST BE RE-FOUND FROM THE GATES.

Kind (f1)	Precision	Recall	F1
hard_add	0.985	1.000	0.993
hard_mul	1.000	1.000	1.000
ff_en	–	–	0.364
register	–	–	0.021
mux	–	–	0.018
mux_bit	–	–	0.081
macro-F1 (all kinds)			0.354
Kind (f2)			
adder	1.000	0.134	0.237
subtractor	1.000	0.074	–
mux_bit	–	–	0.100
adder_cin	–	–	0.029
macro-F1 (all kinds)			0.108

TABLE II
L2 CLUSTERING AGAINST THE KNOWN MODULE HIERARCHY (68 DESIGNS, MEAN OVER THE RENAMED FLOORS, IDENTICAL TO BASE). MODULARITY AND NMI(MODULE_PATH) PER METHOD.

Method	Modularity	NMI
Louvain	0.4220	0.1360
Spectral	0.3620	0.1378
Affinity propagation	-0.2264	0.2708
k-means	-0.2912	0.3220

methods. 271 of 272 partition records are identical between base and renamed floors; the one difference exposed a determinism defect in the spectral path (repeated runs on the same base design give modularities -0.0079, -0.0410 and 0.0252), reported as an open issue rather than hidden.

The GNN result answers the paper's question directly. A node-only GraphSAGE loses to the 1-WL baseline on real micros (macro-F1 0.2413 against 0.2711); combining node, graphlet, WL and NPN features turns it around (0.3545 against 0.2711 on the earlier real-micro run), and the full labelled run of this experiment gives test macro-F1 0.4241 against 0.2717 (accuracy 0.8863 against 0.8492, 3,932 test nodes across 13 designs). The features carry the recovery, not the message passing alone.

C. The Gate-Breaker metrics

Gate-Breaker's metrics compare a recovered RTL against a reference: Success Rate, Success@k, AST similarity and CFG similarity. Loki recovers structure, not a new RTL text, so the direct comparison is the replication: the Gate-Breaker harness itself, run on the simple benchmarks with the only language model available on the host, gave success rate 0.933 on Q1 and 0.829 on Q2, Success@10 1.000 in both quartiles, AST similarity 0.576 and 0.444, CFG similarity 1.000 and 0.736, with 169 of 174 extracted candidates functionally correct – a first replication on the fallback micro set, not the paper's VerilogEval set. The VerilogEval replication, on the quartiles the paper used, is reported in Table III with the AST and CFG kernels recomputed with common-width feature vectors (the first run padded per graph and truncated in the zip, which the v1.1.0 CFG probe found; those columns are not comparable to it).

TABLE III
GATE-BREAKER METRICS ON THE VERILOGEVAL QUARTILES (ONE RUN, K=10, HOST MODEL), DELTAS AGAINST THE PAPER'S O3-MINI.

Quartile	SR	S@1	S@5	
Q1	**[TODO: RUNNING]**	**[TODO: RUNNING]**	**[TODO: RUNNING]**	**[T**
Q2	**[TODO: RUNNING]**	**[TODO: RUNNING]**	**[TODO: RUNNING]**	**[T**

What the two lines of work measure is complementary. The language model recovers functional behaviour from the gates and names nothing reliably; Loki recovers the structures the flow preserved (hard blocks, the hierarchy, the signal roles its features see) and proves what it claims. The Gate-Breaker future work – bus naming, datapath/control classification, partitioning – is exactly L2's table, and its numbers are the baseline a model-assisted Loki would have to beat.

V. DISCUSSION

Three observations follow. First, *the answer depends on what the flow erased*. The structures that survived synthesis recover almost perfectly without any intelligence beyond reading the pass's own facts; the structures synthesis erased (the counter behind its flip-flop, the mux behind its gates) recover weakly, and no amount of honest Bayesian machinery changes that without better detectors or a model. Second, *the graph features carry the recovery*: the GNN only beats 1-WL when node, graphlet, WL and NPN features combine, which is the same lesson as the clustering table (community methods find the hierarchy, centroid methods do not). Third, *the benchmarks made the limits visible*: the renamed-instance protocol proved naming neutrality over 105,646 identifiers, and the same protocol exposed a rename-script defect (library `.blackbox` model names were renamed, which silently removed every hard proposal on renamed floors) and the spectral determinism defect – both fixed or reported, and both invisible without the protocol.

VI. CONCLUSION

Loki III asks how far netlist-to-RTL reverse engineering gets without a language model. On the benchmarks of this paper the answer is: the already-reverse-engineered structures, perfectly; the rest, weakly but measurably, with calibration that reports its own error, a graph stage whose features beat 1-WL, and clusters that carry the designer's hierarchy. The methodology – provenance-tracked labelled datasets plus the renamed-instance protocol – is the part we would export to any future comparison, with or without a model.